

Week 16 Mini-Project · Build the Evaluable Core

Build the thin slice you specced last week, then prove it works with numbers instead of vibes.

Theme	Applied ML engineering. Build a working RAG / agent slice and an evaluation + observability harness so quality becomes a measured number.
Effort	About 1.5 to 2 days. The slice is intentionally thin; the eval tools auto-generate most metrics.
New tools	Ragas (RAG metrics + golden-set generation) and Langfuse (tracing, observability, scoring). Tier A adds DeepEval (pytest CI gate). langfuse.com · <code>pip install ragas langfuse deepeval</code>
Also use	Python, your LLM API, Chroma, text-embedding-3-small.
You ship	Working slice · golden eval set · baseline scorecard · 2 to 3 experiments · live Langfuse traces · one-page executive memo · updated repo.
Interview signal	The applied ML round: "how do you know it works?", "how do you evaluate a RAG system?", plus Python internals.

How this connects to your capstone

This is your capstone's first working vertical slice with a measurement harness bolted on. The golden set and KPIs you built in Week 15 become the test bed here. The architecture you drew becomes a traced pipeline in Langfuse. By Friday you own the one thing that separates an ML engineer from a prompt tinkerer: a system whose quality you can defend with a scorecard. That scorecard is also direct fuel for this week's mock interview.

The scenario

Same **Northwind Support Copilot** (or your capstone). Last week you defined what trustworthy means. This week you build the slice and instrument it so you can show, with numbers, that it retrieves the right context and does not hallucinate.

Why this matters

A RAG or agent demo that "looks good" is worth nothing until you can prove it stays grounded and fetches the right context. Eyeballing outputs stops working past a dozen examples. The teams that ship instrument their systems: they measure faithfulness, relevance, and retrieval quality against a fixed test set on every change. This week you build that harness.

Core mission (Tier C)

1. Build the thin vertical slice (Python)

ingest → chunk → embed (text-embedding-3-small) → Chroma → retrieve top-k → assemble prompt with retrieved context → LLM answer with citations. Keep it small; it must run end to end on your 15 to 30 docs from Week 15.

2. Build a golden eval set (30 to 50 Q&A pairs)

Each row: question, ideal answer, and the source document(s) that should be retrieved. Cover four flavors: easy, ambiguous, multi-hop (needs two documents), and adversarial (answer is not in the corpus, so the system should say "I do not know"). You may draft with the Ragas synthetic generator, then **hand-verify every row**; raw synthetic data fails.

3. Measure with Ragas

Compute the four core metrics on your golden set:

Metric	What it asks	Reference target
Faithfulness	Does every claim trace back to retrieved context (no hallucination)?	≥ 0.90 to ship; < 0.70 unsafe
Answer relevancy	Does the answer actually address the question?	≥ 0.80
Context precision	Are the retrieved chunks relevant, and in good order?	≥ 0.70
Context recall	Does the retrieved context contain the answer?	≥ 0.80

Produce a **baseline scorecard**: metric → your score → your Week 15 target → pass / fail. Use your own Week 15 targets as the bar; the numbers above are reference points.

4. Instrument with Langfuse

Wrap your pipeline with the `@observe()` decorator so each run logs a trace (retrieval step, generation step, tokens, latency, cost). Put one screenshot of a trace and one of the dashboard in your README.

5. Run 2 to 3 improvement experiments

Change **one** thing each (top-k, chunk size, a reranker, or the prompt), re-measure on the same golden set, and report the metric delta. State which change you keep and why.

Depth ladder

Higher tiers include everything below. Finish your tier, then attempt the Bonus.

Tier C • Core. Working slice + golden set (30+ pairs) + Ragas baseline scorecard on the four core metrics + live Langfuse tracing + one experiment with a measured delta.

Tier B • Stretch. Add adversarial / out-of-corpus handling (the system abstains; measure abstention correctness); a citation-accuracy check (does the cited chunk actually support the claim); the bounded tool / action from your design, traced in Langfuse; and an error-analysis table of your five worst cases with root cause tagged as retrieval or generation.

Tier A • Advanced. Add a DeepEval pytest gate that fails the build when faithfulness drops below your floor (paste the test and a screenshot of it passing or failing); a latency and cost-per-query reading pulled from Langfuse; and a short "before production" note (monitoring, a freshness / re-index plan, guardrails).

Bonus. Add reranking or hybrid (keyword + vector) retrieval and quantify the lift; or run an LLM-as-judge comparison of two prompts using Langfuse experiments.

What "done" looks like (Tier C)

A reviewer clones your repo, runs one command, and reproduces a scorecard showing faithfulness, answer relevancy, context precision, and context recall on your golden set. Langfuse shows live traces with token and latency data. One experiment shows a before / after metric change with a one-paragraph explanation. Every number ties back to a target you set in Week 15.

LLM-integrated task (pass / fail gate)

Take your single worst-scoring question (lowest faithfulness, or a retrieval miss). Ask an LLM to diagnose why it failed and propose a fix. In your repo, paste: the failing case (question, retrieved context, bad answer, scores), the prompt, the model's diagnosis, **your** critique (was the diagnosis right? did the fix actually move the metric when you tried it?), and the result. A diagnosis with no attempted fix and re-measurement fails the gate. Failing caps your score at Tier C until you resubmit.

Reflection (answer in README, 3 to 5 sentences each)

- Where did your system fail most: retrieval or generation? How do the metrics tell you which?
- Did any metric look good while the answer was actually bad? What does that teach you about single-metric thinking?
- Which Week 15 target did you hit, miss, or have to revise after seeing real numbers?

Executive memo (one page, interview-ready)

The slice you built, the baseline scorecard, the experiment that moved a metric the most, and your honest read on what is and is not production-ready. This is your answer to "tell me about a time you evaluated an AI system." Save as `EXEC_MEMO.md`.

Submission

Push to the same repo as Week 15, with new folders:

```
/app /eval (golden set + Ragas) /experiments /observability (Langfuse + screenshots) EXEC_MEMO.md README.md
```

Submit the repo link through the LMS. A 5-minute viva within 48 hours may ask you to interpret your own scorecard; a scorecard you cannot read does not pass.

Tool pointers (learn fast)

Langfuse: free cloud signup or self-host in minutes; add the `@observe()` decorator and the OpenAI integration to auto-capture traces; a 10-minute walkthrough and a quickstart notebook exist. **Ragas:** `pip install ragas`; feed it your questions, answers, contexts, and ground-truth to get the four metrics; it can also generate a synthetic golden set you then verify. **DeepEval:** `pip install deepeval`; `pytest-native`, so a faithfulness gate is a normal test that your CI can run.